

NOCTURNE BLOOD RUN

AYRINTILI PROJE RAPORU

Unity Scripting - Visualization Basics Kapsamında Teknik ve Gorsel Inceleme

Ogrenci	Aysenisa Yasar
Numara	222511022
Tarih	20.05.2026
Proje	Nocturne Blood Run

Bu rapor, Unity ile geliştirilen Nocturne Blood Run projesinin teknik altyapısını, sahne kurgusunu, script organizasyonunu ve ders kapsamında talep edilen Unity kavramlarını ayrıntılı biçimde incelemek amacıyla hazırlanmıştır.

Raporun Amacı

Projede kullanılan her zorunlu Unity kavramını somut kod örnekleri ve dosya referanslarıyla açıklamak. Oyunun sahne kurulumunu, oynanış akışını ve scriptler arası veri akışını görsellerle desteklemek. Uygulamanın neden bu yapıda tasarlandığını ve hangi teknik seçimlerin yapıldığını değerlendirmek.



Gorsel 1. Kasaba, orman, baslangic noktalar, altinlar ve kacis rotalarinin ustten sematik yerlesimi.

1. Yönetici Özeti

Nocturne Blood Run, Unity üzerinde geliştirilmiş iki oynanabilir karakterli bir kasis ve hayatta kalma prototipidir. Oyunun ana fikri, karanlık bir kasaba ve onu çevreleyen sonbahar ormanında ilerleyerek altın toplamak, yeterli skor sonrasında silah kilidini açmak ve oyuncuları avlayan canavarı etkisiz hale getirmektir.

Proje yalnızca oynanış mantığını değil, sahne kurulumunu da otomatikleştiren bir editör aracı ile oluşturulmuştur. Bu sayede ortam prefablarından animasyon denetleyicilerine, NavMesh kurulumundan HUD ve buton yapısına kadar pek çok öğenin tekrar üretilebilir olması sağlanmıştır.

- Remy ve Peasant Girl adlı iki karakter, farklı kontrol semaları ile aynı sahnede aynı anda yönetilebilir.
- Canavar, NavMeshAgent ve görüş tabanlı RayCast kontrolü ile oyuncuları kovalar.
- Altın toplama, skor sistemi, silah kilidi, ateş etme, oyun sonu ve yeniden başlatma akışları tek bir Director scripti tarafından koordine edilir.
- SceneBuilder tarafından üretilen prefablar ve sahne elemanları, raporun merkezindeki Unity kavramlarının büyük çoğunluğunu doğrudan içermektedir.

Bu Raporda Neler Var?

Projenin konusu ve sahne dili

Script mimarisi ve veri akış diyagramı

10 zorunlu Unity kavramının tamamına ait ayrıntılı analiz

Dosya ve satır referansları ile teknik değerlendirme

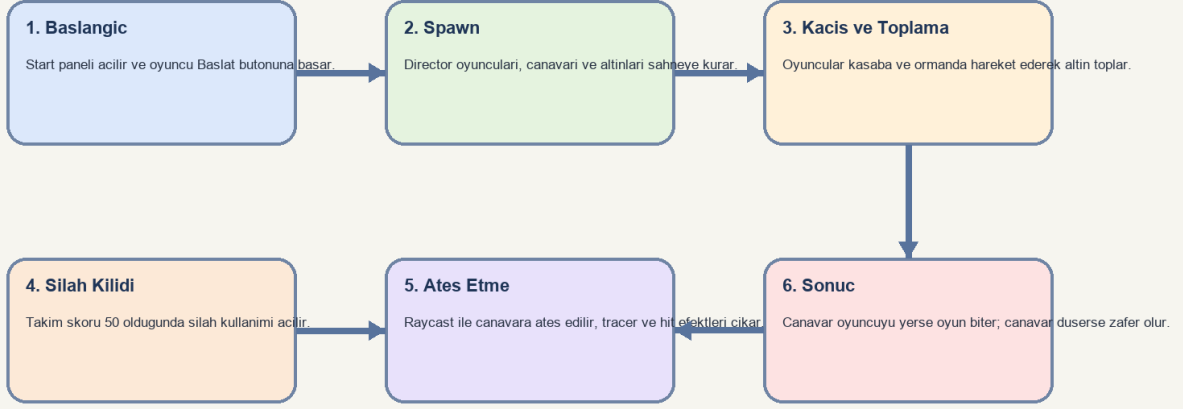
2. Projenin Konusu, Hedefi ve Oynanış Kurgusu

Oyunda iki insan karakteri, canavarın saldırısından kurtulmak için kasabanın ana yolunu, plaza alanını ve orman yollarını kullanarak hareket eder. Başlangıçta oyuncular sadece kasis ve altın toplama davranışına sahiptir. Takım skoru gerekli esige ulaştığında ise ateş etme yeteneği acilir ve oyun savunmadan saldırıya doğru evrilir.

Bu kurgu, ders kapsamında beklenen birden fazla Unity kavramını tek bir hikaye içerisinde toplamaya imkan vermiştir. Altınlar Trigger ile toplanmakta, oyuncu hareketi NavMesh üzerinde ilerlemekte, canavar RayCast ile hedefini görmekte ve partiküller hem ortam hem de etkileşim geri bildirimi için kullanılmaktadır.

1. Oyun butonla başlar ve Director senaryoyu aktif eder.
2. Oyuncular WASD ve yön tuşları ile hareket ederek altın toplar.
3. Skor 50 olduğunda silah kullanımı acilir.
4. Oyuncular RayCast tabanlı ateş ile canavarı hasarlayabilir.
5. Canavar oyuncuya yetişirse oyun biter; canavar düşürülürse zafer ekranı oluşur.

Nocturne Blood Run - Oynanış Akış Diyagramı



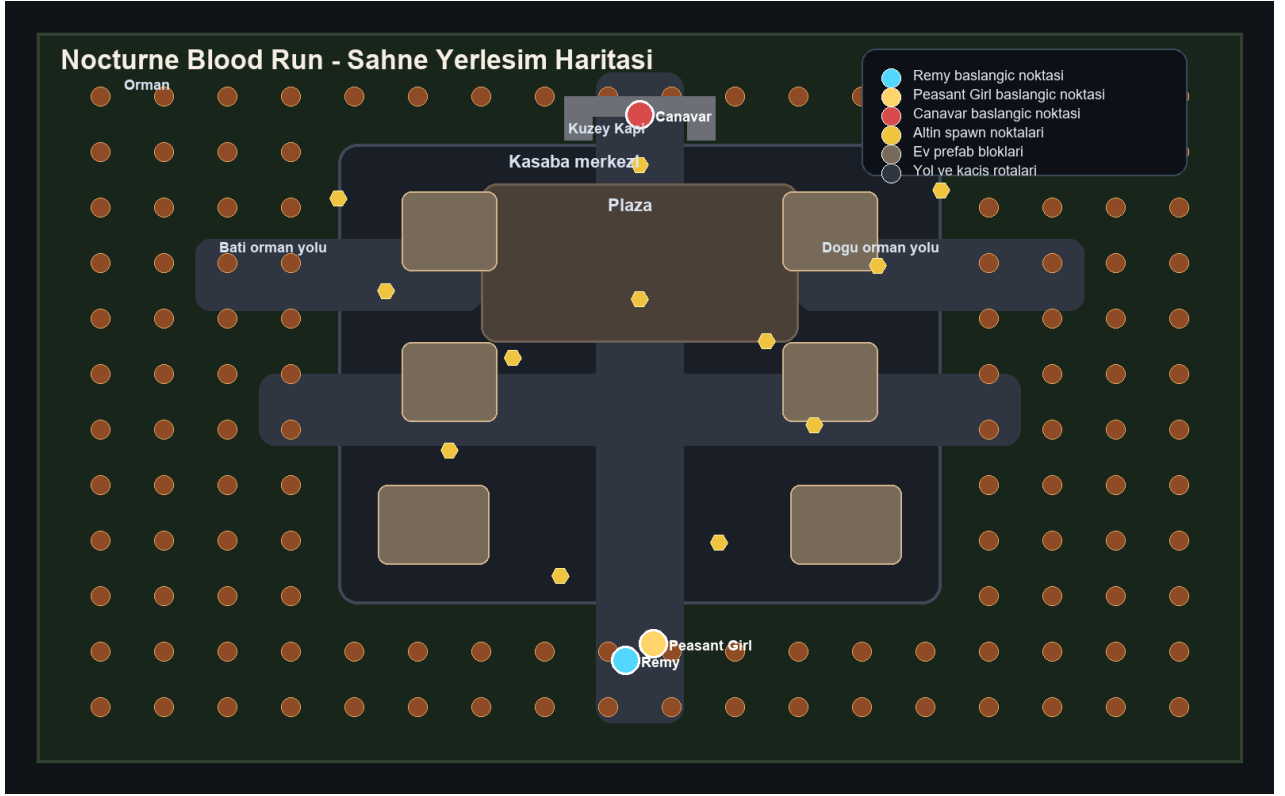
Akış mantığı: butonla başlayan oyun önce spawn ve kontrol aşamasına geçer, ardından altın toplama ve silah kilidi ile ilerler. Canavarın yenilmesi ya da oyuncuların yenilmesi son durumu belirler.

Gorsel 2. Oyunun başlangıçtan sonuca kadar ilerleyen temel oynanış akışı.

3. Gorsel Tasarım ve Sahne Cozumlemesi

Sahne tasarımı, karanlık bir kasaba ile onu çevreleyen sonbahar ormanının karsılığı üzerine kurulmuştur. Merkezde yol ve plaza gibi daha okunaklı geometri kullanılırken, çevredeki orman daha yoğun ağaç tekrarları ile kapatılmıştır. Bu yapının amacı, hem kaçış rotalarını netleştirmek hem de tehdit hissini arttırmaktır.

- Kasaba merkezi: ana yol, plaza, evler, lambalar ve kapıdan oluşan kontrollü oynanış omurgası.
- Orman katmanı: yan kaçış imkanı sunan ancak görüş ve takip baskısını arttıran bir çevre.
- Işık düzeni: karanlık tonlu arka plan üzerine mavi-gri moonlight ve sıcak lamba ışıkları.
- Partikül atmosferi: düşük yoğunluklu yaprak sürüklenmesi ve zemin sisi ile sonbahar havası.



Gorsel 3. Oyun alanı; kasaba merkezi, orman bantları, altın noktalar ve canavar başlangıcı ile birlikte sematik olarak görülmektedir.

Sahne Tasarımında One Cikan Teknik Secimler

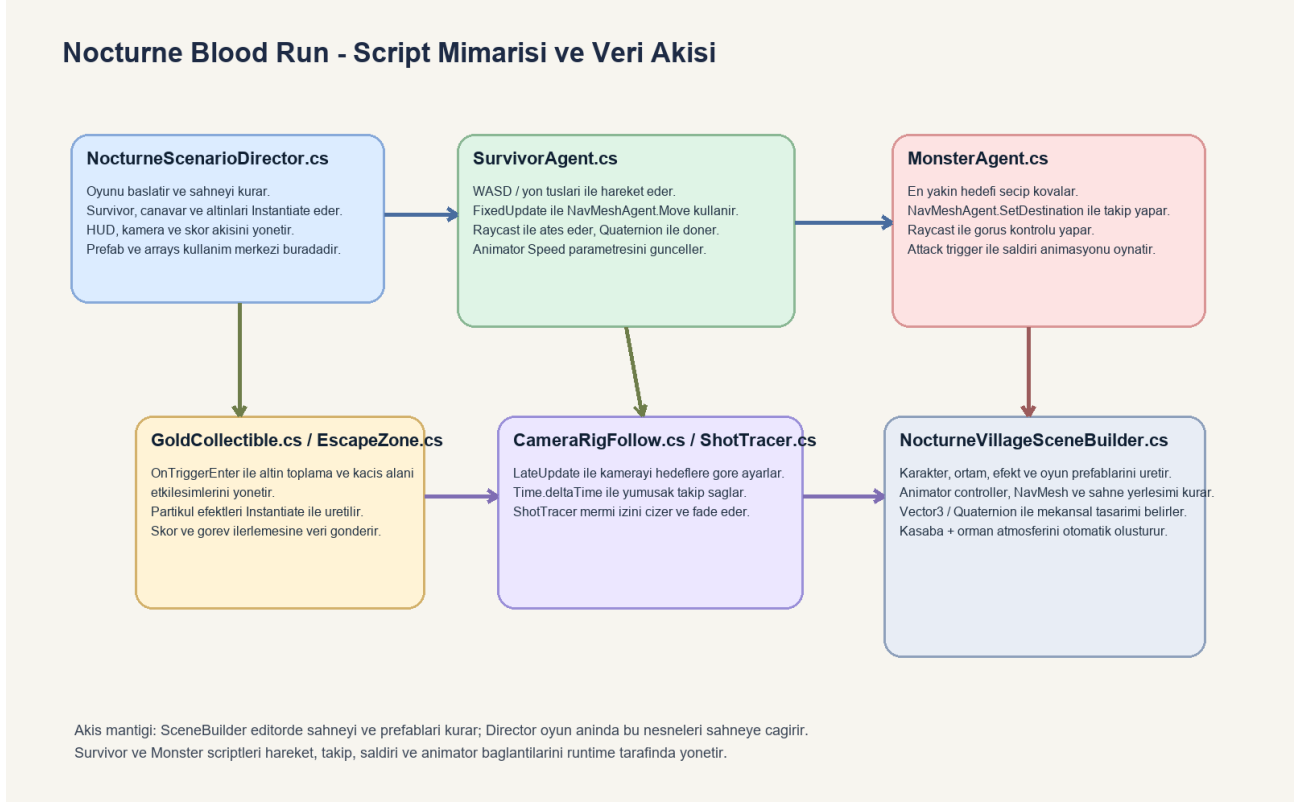
SceneBuilder yolu, curb elemanlarını, evleri ve orman nesnelerini editörde otomatik üretir.

Altın noktalarının elde tek tek yerleştirilmesi yerine Vector3[] dizisi ile toplu oluşturma tercih edilmiştir.

Atmosfer efektleri yolun tam ortasını kapatmayacak şekilde kenarlara ve kritik noktalara dağıtılmıştır.

4. Script Mimarisi ve Veri Akisi

Projede sorumluluklar farklı scriptlere bölünmüştür. Bu dağılım, kodun okunabilirliğini ve yeni özellik ekleme kolaylığını artırır. Özetle SceneBuilder editörde sahneyi üretir; NocturneScenarioDirector oyun anında senaryoyu başlatır; SurvivorAgent ve MonsterAgent bireysel davranışları yönetir; CameraRigFollow ve ShotTracer ise yardımcı görüntü katmanını destekler.



Gorsel 4. Script bazlı sorumluluk dağılımı ve oyun içindeki veri akisinin özet diyagramı.

Dosya	Ana Sorumluluk	Dikkat Çeken API veya Yapi
NocturneScenarioDirector.cs	Spawn, HUD, skor ve oyun durumu yönetimi	Instantiate, arrays, buton akisi
SurvivorAgent.cs	Oyuncu hareketi, ateş etme, animator surumu	FixedUpdate, RaycastAll, Quaternion
MonsterAgent.cs	Takip, görüş kontrolü ve saldırı	NavMeshAgent, Raycast, Animator Trigger
CameraRigFollow.cs	Kameranin hedeflere göre hareketi	LateUpdate, deltaTime, Transform[]
GoldCollectible.cs	Altın animasyonu ve toplama	OnTriggerEnter, Instantiate
EscapeZone.cs	Kaçış alanına giriş kontrolü	OnTriggerEnter
ShotTracer.cs	Mermi izinin fade edilmesi	LineRenderer, Time.deltaTime
NocturneVillageSceneBuilder.cs	Prefab ve sahne otomasyonu	PrefabUtility, AnimatorController, BuildNavMesh

5. Ders Kriterleri ve Karsilanma Durumu

Asagidaki tablo, ders gereksinimlerinde istenen tum Unity basliklarinin projede karsilandigini ve bunlarin hangi scriptlerde yer aldigini ozetlemektedir.

Baslik	Durum	Ana Dosyalar	Not
Prefab	Var	SceneBuilder, Director	Karakter, ortam, efekt ve gameplay nesneleri prefab olarak kurulmustur.
Vector3 / Quaternion	Var	SurvivorAgent, CameraRigFollow, SceneBuilder	Hareket, bakis, mermi cikisi ve sahne yerlesimi bu yapiyla hesaplanir.
Partikul efektleri	Var	SceneBuilder, GoldCollectible, MonsterAgent	Spawn, vurulma, altin ve atmosfer efektleri vardir.
Instantiate	Var	Director, SurvivorAgent, EscapeZone	Runtime nesne uretimi aktif sekilde kullanilir.
RayCast	Var	SurvivorAgent, MonsterAgent	Hem ates hem de canavar gorusu icin kullanilir.
FixedUpdate / LateUpdate	Var	SurvivorAgent, CameraRigFollow	Fizik ve kamera mantigi farkli update dongulerine ayrilmistir.
Trigger / Collision	Var	GoldCollectible, EscapeZone	Trigger tabanlı etkileşim tercih edilmistir.
Arrays	Var	Director, CameraRigFollow, SceneBuilder	Spawn noktası ve hedef listeleri dizilerle tutulur.
NavMeshAgent	Var	SurvivorAgent, MonsterAgent, SceneBuilder	Takip ve hareketin temel yol bulma katmanidir.
Animator	Var	SceneBuilder, SurvivorAgent, MonsterAgent	Idle, run, walk ve attack gecisleri kontrol edilir.

6. Zorunlu Unity Kavramlarının Ayrıntılı Analizi

Bu bölümde, ders listesinde istenen on temel Unity başlığı tek tek incelenmektedir. Her kavram için önce ne olduğu açıklanmakta, daha sonra Nocturne Blood Run projesinde hangi kod bloklarında kullanıldığı ve bu seçimin oynanışa ne kattığı anlatılmaktadır.

6.1 Prefab Kullanımı

Kavramsal açıklama:

Prefab, bir oyun nesnesinin ayarlarıyla birlikte tekrar kullanılabilir şablon olarak saklanmasıdır. Unity projelerinde aynı tip nesneyi tekrar tekrar kullanırken tutarlılık ve hız sağlar.

Projede nasıl kullanıldı:

Projede prefab mantığı iki katmanda kullanılmıştır: birincisi SceneBuilder tarafında karakter, ev, ağac, lamba, altın ve efekt prefablarının üretilmesi; ikincisi Director tarafında bu prefabların runtime sırasında Instantiate edilerek sahneye yerleştirilmesidir.

Kod referansları:

- NocturneVillageSceneBuilder.cs 358-381: efekt ve gameplay prefablarının oluşturulması.
- NocturneVillageSceneBuilder.cs 426-528: survivor ve monster prefablarının Animator, NavMeshAgent ve gameplay scriptleriyle birleştirilmesi.
- NocturneScenarioDirector.cs 199-238: prefabların oyun anında spawn edilmesi.

Projeye katkisi:

- Aynı nesne yapısının birden fazla noktada tutarlı biçimde kullanılmasını sağlar.
- Sahne kurulumunu hızlandırır ve script tabanlı otomasyonun temelini oluşturur.
- Karakter veya efekt değişikliği gerektiğinde tek bir kaynaktan güncelleme imkanı verir.

Teknik Not

Bu projede prefab kullanımı sadece nesne saklamak için değil, sahneyi otomatik üreten editör akışını da besleyen ana mekanizma olarak seçilmiştir.

6.2 Vector3 ve Quaternion Kullanımı

Kavramsal açıklama:

Vector3, Unity sahnesindeki konum, yön ve ölçek hesapları için kullanılan temel veri yapısıdır. Quaternion ise acısal dönüşleri gimbal lock sorunu olmadan yönetmek için kullanılır.

Projede nasıl kullanıldı:

Oyuncu hareketi, kameranın hedeflere bakması, mermi çıkış noktası, altın ve spawn pozisyonları ile sahnedeki yol-ev-orman yerleşimleri bu iki yapıyla kurulmuştur.

Kod referansları:

- SurvivorAgent.cs 100-126: hareket yönünün Vector3 ile hesaplanması ve Quaternion.Slerp ile dönüş.
- SurvivorAgent.cs 303-309: mermi çıkışı için origin ve hitPoint hesapları.
- CameraRigFollow.cs 58-63: desiredPosition ve desiredRotation hesapları.
- NocturneVillageSceneBuilder.cs 602-643, 736-911: sahne nesnelerinin koordinat ve rotasyonları.

Projeye katkisi:

- Oyuncu hareketinin yönel olarak doğru ve akıcı olmasını sağlar.
- Kamera takibinin merkez noktasına göre stabil davranmasına yardımcı eder.
- Sahne içindeki tüm mekansal organizasyonu script ile tekrar üretilebilir hale getirir.

6.3 Partikül Efektleri

Kavramsal aciklama:

Partikul sistemleri, oyundaki olaylari gorsel olarak vurgulamak ve atmosfere hareket katmak icin kullanilir. Patlama, duman, sis, yaprak ve isik serpintisi gibi pek cok kucuk efekti verimli sekilde uretir.

Projede nasil kullanildi:

Bu projede spawn sirasinda mist, vurulma sirasinda impact ve blood burst, ates sirasinda muzzle flash, altin toplanirken gold burst, ortamda ise autumn drift ve ground fog kullanilmistir.

Kod referanslari:

- NocturneVillageSceneBuilder.cs 358-372: tum efekt prefablarinin uretildigi bolum.
- NocturneVillageSceneBuilder.cs 1614-1703: sonbahar yapragi ve zemin sisi gibi atmosfer efektlerinin ayarlanmasi.
- GoldCollectible.cs 52-54: altin pickup efekti.
- MonsterAgent.cs 220 ve 255: saldiri ve olum anlarindaki efektler.

Projeye katkisi:

- Oyuncuya olaylari sonucunu aninda anlatir.
- Karanlik kasaba temasini gorsel olarak destekler.
- Spawn, ates ve hasar gibi olaylari geri bildirim kalitesini artirir.

Teknik Not

Partikuller yolun ortasinda gorusu kapatmayacak bicimde konumlandirilmis; bu secim oyun okunurlugunu korumak icin yapilmistir.

6.4 Instantiate Kullanımı

Kavramsal açıklama:

Instantiate, bir prefab veya nesnenin çalışma anında yeni bir kopyasını üretmek için kullanılır. Dinamik oyunlarda sabit sahne kurulumuna bağlı kalmadan yeni nesne yaratmaya imkan verir.

Projede nasıl kullanıldı:

Projede oyuncular, canavar, altınlar, mermi izi ve ani efektlerin tamamı runtime üretim mantığı ile oluşturulur. Bu sayede oyun tekrar başladığında sahne tekrar kurulabilir.

Kod referansları:

- NocturneScenarioDirector.cs 199-238: survivor, canavar ve altınların oluşturulması.
- SurvivorAgent.cs 309-339: muzzle flash, impact burst ve shot tracer nesnelerinin oluşturulması.
- EscapeZone.cs 35 ve GoldCollectible.cs 54: etkileşim anlarında efekt üretimi.

Projeye katkisi:

- Oynanışın durumuna göre nesne üretimine izin verir.
- Sahneyi tek seferlik değil, tekrar oynanabilir hale getirir.
- Efektlerin yalnızca gerektiğinde ortaya çıkıp daha hafif bir sahne akışı sağlamasına yardımcı eder.

6.5 RayCast Kullanımı

Kavramsal açıklama:

RayCast, sahnede bir doğrultuda ışık benzeri çizgi gönderip ilk çarpışma noktasını tespit etmeye yarar. Atış mekanikleri, hedef algılama ve görüş kontrolü gibi durumlarda oldukça yaygındır.

Projede nasıl kullanıldı:

Projede oyuncuların ateş sistemi RaycastAll ile, canavarın oyuncuyu görme mantığı ise Raycast ile kurulmuştur. Böylece hem ofansif mekanik hem de AI algılama aynı Unity mantığıyla desteklenmiştir.

Kod referansları:

- SurvivorAgent.cs 312-326: RaycastAll ile merminin çarpıştığı ilk geçerli hedefin bulunması.
- SurvivorAgent.cs 323-326: vurulan collider içinden MonsterAgent referansının alınması.
- MonsterAgent.cs 178-182: canavarın doğrudan oyuncuyu görüp görmediğini test eden Physics.Raycast.

Projeye katkisi:

- Ateş mekanizmine fizik temelli bir mantık kazandırır.
- Canavarın her zaman değil, yalnızca görüş alabildiğinde agresif davranmasını sağlar.
- Sahne engellerinin oynanışa etkide bulunmasına imkan verir.

Teknik Not

RayCast kullanımı, oyundaki tehdit algısını güçlendirir çünkü canavar sadece mesafe değil, görüş doğrulaması da yapmaktadır.

6.6 FixedUpdate, LateUpdate ve DeltaTime

Kavramsal açıklama:

Unity'de farklı update döngüleri farklı sorumluluklar için tercih edilir. FixedUpdate daha kararlı fizik mantığı sunar; LateUpdate ise diğer nesneler hareket ettikten sonra kamera gibi bağımlı sistemleri güncellemek için uygundur. DeltaTime ve fixedDeltaTime hareketleri kare hızından bağımsız yapar.

Projede nasıl kullanıldı:

Projede oyuncu hareketi FixedUpdate icine alinarak agent.Move hesapları sabit zaman adimiyla yapilmistir. Kamera ise LateUpdate icinde karakterlerin son pozisyonlarını izler. Altin animasyonu ve tracer fade davranisi Time.deltaTime ile yumusatilmistir.

Kod referansları:

- SurvivorAgent.cs 93-126: FixedUpdate ile fizik uyumlu hareket ve donus.
- CameraRigFollow.cs 19-63: LateUpdate ile kamera merkezleme ve bakis hesapları.
- GoldCollectible.cs 26-27: delta time kullanarak donme ve bobbing animasyonu.
- ShotTracer.cs 18-33: elapsed += Time.deltaTime ile iz efekti sönmesi.

Projeye katkisi:

- Farkli sistemlerin dogru zamanda guncellenmesini sağlar.
- Kare hizi degisse bile hareketin tutarlılığını korur.
- Kameranın oyuncudan geri kalmadan pürüzsüz izleme yapmasına yardım eder.

6.7 Trigger Kullanimi

Kavramsal aciklama:

Trigger collider, fiziksel carpismadan ziyade bir alanin icine girildigini tespit etmek icin kullanilir. Oyunlarda toplama, bolgeye giris ve gorev tetikleme gibi senaryolar icin idealdir.

Projede nasil kullanildi:

Bu projede altin toplama ve kacis alan mantiklari OnTriggerEnter uzerinden kurulmustur. Collision tercih edilmemistir cunku burada fiziksel itme degil, olay tetikleme davranisi istenmektedir.

Kod referanslari:

- GoldCollectible.cs 17-21: SphereCollider icin isTrigger ayari.
- GoldCollectible.cs 37-58: altina dokunan survivor icin toplama akisi.
- EscapeZone.cs 17-20: bolgenin trigger olarak ayarlanmasi.
- EscapeZone.cs 23-35: survivor kacis alanina girince ReachSafety akisinin cagrilmasi.

Projeye katkisi:

- Toplama ve bolgesel etkileşimleri fizik yiginina gereksiz yuk bindirmeden cozer.
- Oyuncu hareketini bozmadan olay bazli tetikleme yapar.
- Skor, gorev ve oyun sonu mantiklariyla temiz sekilde baglanir.

Teknik Not

Bu projede Collision yerine Trigger secilmesi bilincli bir tasarim karardir; cunku altinlar oyuncuyu itmemeli, sadece algılanmalıdır.

6.8 Arrays Kullanimi

Kavramsal aciklama:

Diziler, ayni tipte birden fazla veriyi sabit sirali yapida saklamaya yarar. Oyun gelistirmede spawn noktalarini, hedef listelerini ve toplu islenecek nesneleri yönetmek icin verimli bir cozumdur.

Projede nasil kullanildi:

Projede oyuncu prefab dizileri, spawn noktasi dizileri, altin noktasi dizileri ve kamera hedef dizileri sistematik bir kurulum mantigi olusturur. SceneBuilder tarafinda da Vector3[] dizileriyle ortam yerlestirmesi yapilmaktadir.

Kod referanslari:

- NocturneScenarioDirector.cs 10-27: survivorPrefabs, survivorSpawnPoints, goldSpawnPoints ve spawnedSurvivors tanimlari.
- NocturneScenarioDirector.cs 193-243: diziler uzerinden dongu ile senaryo kurulumu.
- CameraRigFollow.cs 12-16: takip hedeflerinin Transform[] olarak atanmasi.
- NocturneVillageSceneBuilder.cs 630-643 ve 905-927: altin, drift ve fog pozisyon dizileri.

Projeye katkisi:

- Tek tek nesne atamak yerine toplu yonetim saglar.
- Sahne buyudukce bile kodun duzenli kalmasina yardim eder.
- Yeni spawn noktasi eklemeyi veya mevcutlari degistirmeyi kolaylastirir.

6.9 Navigation Mesh ve NavMeshAgent Kullanimi

Kavramsal aciklama:

NavMesh, sahnede yurunebilir alanlari temsil eden bir navigasyon katmanidir. NavMeshAgent ise bu alan uzerinde hareket eden ve engelleri dolasan yapay zeka veya karakter bilesenidir.

Projede nasil kullanildi:

Projede hem survivor hem de canavar prefablarına NavMeshAgent eklenmistir. Survivor tarafında hareket dogrudan agent.Move ile surulur; canavar tarafında ise SetDestination kullanilarak hedef takip edilir.

Kod referanslari:

- NocturneVillageSceneBuilder.cs 450-456: survivor prefab agent ayarlari.
- NocturneVillageSceneBuilder.cs 499-505: monster prefab agent ayarlari.
- NocturneVillageSceneBuilder.cs 716: BuildNavMesh ile sahnenin navigasyon verisinin olusturulmasi.
- SurvivorAgent.cs 122: agent.Move ile kontrollu hareket.
- MonsterAgent.cs 97-103: SetDestination ile hedef kovalaması.

Projeye katkisi:

- Canavarın kasaba ve orman içinde gerçekçi şekilde yol bulmasını sağlar.
- Oyuncu hareketini grid bağımlı olmaktan çıkarır ve sahne topolojisine uyarlar.
- Sahne büyük olduğu için manuel rota mantığı yazma ihtiyacını azaltır.

Teknik Not

NavMesh kullanımı, orman ve kasaba gibi farklı geometri alanlarında davranış kararlılığını belirgin biçimde arttırmıştır.

6.10 Animation ve Animator Kullanimi

Kavramsal aciklama:

Animator sistemi, bir karakterin farklı animasyon klipleri arasında koşullu geçişler yapmasını sağlar. Hangi animasyonun ne zaman oynatılacağı parametrelerle kontrol edilir.

Projede nasil kullanildi:

Projede survivor karakterleri için Idle ve Run geçişleri, canavar için ise Idle, Walk ve Attack durumları tanımlanmıştır. Hareket hızına göre Speed parametresi güncellenir; saldırı anında Attack trigger'i ateslenir.

Kod referanslari:

- NocturneVillageSceneBuilder.cs 270-335: Animator Controller ve state machine oluşturma.
- NocturneVillageSceneBuilder.cs 443-445 ve 492-494: runtime animator controller bağlantıları.
- SurvivorAgent.cs 353-360: Speed parametresinin hareket büyüklüğüne göre güncellenmesi.
- MonsterAgent.cs 206-208 ve 269: Attack trigger ve Speed set işlemleri.

Projeye katkisi:

- Karakterlerin dururken, yürürken ve koşarken doğal görünmesini sağlar.
- Canavarın saldırı anının görsel olarak anlaşılır olmasına yardımcı eder.
- Oyundaki hareket verisini görsel ifade ile birleştirir.

7. Oyun Akisinin Teknik Yorumlanması

Nocturne Blood Run sadece kavram listelerini gösteren bir demo değil, birbirine bağlı bir oynanış sistemi olarak tasarlanmıştır. Aşağıdaki akis oyunun çalışma mantığını teknik açıdan yorumlamaktadır.

- Oyun açıldığında Director sahneyi hazırlar fakat karakterleri hareketsiz tutar; start paneli oyuncudan buton girdisi bekler.

7. BeginGame çağrıldığında survivor scriptleri aktif hale gelir ve canavar davranışı acilir.
8. Oyuncular altın topladıkça RegisterGoldPickup skor sistemini günceller ve gerekli esikte silah kilidini açar.
9. Silah acıldıktan sonra SurvivorAgent Raycast tabanlı ateş sistemini kullanarak canavara hasar verebilir.
10. MonsterAgent hedefi bulduğunda SetDestination ve Attack coroutine akisi ile saldırı davranışına geçer.
11. Survivor yenilirse Director oyun sonu durumunu yazar; canavar yenilirse zafer akisi tetiklenir.

Akis Tasarımındaki Önemli Nokta

Sistemin merkezi Director olsa da hareket, gorus, saldırı, kamera ve toplama gibi alt davranışlar ilgili scriptlere dağıtılmıştır.

Bu dağılım yeni mekanik eklerken mevcut yapının bozulmamasını kolaylaştırır.

8. Dosya ve Kod Referans Özeti

Aşağıdaki tablo, raporda en çok referans verilen dosyaların hangi görevi üstlendiklerini ve inceleme sırasında neden on plana çıktığını özetler.

Dosya	Katman	Temel Sorumluluk	Rapor İçindeki Önemi
Assets/Editor/NocturneVillageSceneBuilder.cs	Editor	Prefab, sahne, NavMesh ve HUD üretimi	Zorunlu kavramların büyük kısmı editörde kurulan altyapıyla ilişkilidir.
Assets/Scripts/Nocturne/NocturneScenarioDirector.cs	Runtime	Spawn, skor, butonlar, oyun sonu	Tüm oynanış akisini koordine eder.
Assets/Scripts/Nocturne/SurvivorAgent.cs	Runtime	Oyuncu kontrolü ve ateş	Vector3, Quaternion, FixedUpdate, RayCast ve Animator burada bir araya gelir.
Assets/Scripts/Nocturne/MonsterAgent.cs	Runtime	Takip ve saldırı AI	NavMeshAgent, RayCast ve Animator davranışlarını birleştirir.
Assets/Scripts/Nocturne/CameraRigFollow.cs	Runtime	Kamera takibi	LateUpdate ve deltaTime kullanimini gösteren net bir örnektir.
Assets/Scripts/Nocturne/GoldCollectible.cs	Runtime	Altın toplama	Trigger, Instantiate ve animasyon benzeri görsel davranışları içerir.
Assets/Scripts/Nocturne/EscapeZone.cs	Runtime	Kaçış alanının tetiklenmesi	Trigger mantığını ikinci bir örnekle destekler.
Assets/Scripts/Nocturne/ShotTracker.cs	Runtime	Ateş izinin fade edilmesi	deltaTime ile görsel efekt omrunu yönetir.

9. Teknik Değerlendirme ve Sonuç

Nocturne Blood Run projesi, ders kapsamında beklenen Unity konu başlıklarını tek tek göstermek yerine bunları bir oyun döngüsü içinde birleştirdiği için daha güçlü bir uygulama örneğidir. Bir yanda editörde sahne üreten bir otomasyon katmanı, diğer yanda runtime davranışları yöneten scriptler bulunmaktadır. Bu ayırım, projenin hem teknik olarak okunabilir hem de genişletilebilir olmasını sağlar.

- Projenin en güçlü tarafı, prefab-temelli sahne kurulumu ile runtime mekaniklerini aynı yapıda birleştirmesidir.

- NavMeshAgent, RayCast ve Animator secimleri, canavar ile oyuncular arasindaki etkileşimi net ve anlaşılır hale getirir.
- Trigger kullanım kararı, altın toplama ve kasis alanı gibi etkileşimlerde gereksiz fizik karmasasını engeller.
- Partikül efektleri ve atmosfer nesneleri, projenin görsel dilini sadece teknik değil sunumsal olarak da güçlendirir.

Genel Sonuç

Bu proje, Unity'nin temel script ve görselleştirme kavramlarını ders seviyesinde karşılaştırmaya olanak tanıyarak bir mini oyun sistemi halinde sunmaktadır.

Dolayısıyla raporlanan her başlık teorik örnek olmaktan çıkmış, sahnede çalışan bir mekanizmaya dönüşmüştür.